Exception Monitoring Automation – Project Overview Document

1. Overview

This document summarizes the goal, rationale, and planned approach for automating the **Dynatrace Exception Monitoring** process currently performed manually by the support team. The automation will streamline exception extraction, comparison, and alerting activities using a structured workflow built with Power Automate.

The solution is designed to replace repetitive manual steps such as executing Dynatrace queries, downloading reports, updating Excel sheets, identifying new or elevated exceptions, and notifying the appropriate team through Microsoft Teams.

---

2. Problem Statement

The current monitoring process involves several manual steps performed repeatedly during deployments (every 30 minutes) and regular business days (every 1 hour). These include:

- Running exception monitoring queries in Dynatrace.
- Downloading the results as CSV files.
- Copying and pasting the data into a baseline Excel sheet.
- Manually checking for new or elevated exceptions.
- Re-checking elevated exceptions after 30 minutes to confirm rising trends.
- Informing the appropriate team through Microsoft Teams.

This process is:

- **Time-consuming**, especially during deployments.
- **Error-prone**, as manual comparison may lead to missed or misclassified exceptions.
- **Inconsistent**, depending on who is performing the checks.
- **Non-scalable**, because exceptions and workloads continue to grow.

Automation is required to eliminate manual intervention, reduce operational risk, and ensure timely detection of critical exceptions.

---

3. Success Criteria

The automation will be considered successful when the following criteria are met:

Functional Success

- Automatically retrieves exception data from Dynatrace via API or structured input.
- Parses and transforms the data into a usable model.
- Detects **new** exceptions (not seen historically).
- Detects **elevated** exceptions (count higher than previous run).
- For elevated exceptions, **re-checks after 30 minutes** and only alerts if values continue to increase.
- Sends accurate, formatted alerts to Microsoft Teams.

Operational Success

- Runs reliably on schedule (30 minutes during deployment / 1 hour daily).
- Requires minimal or no human intervention.
- Produces consistent, repeatable results.

Business Success

- Reduces manual monitoring effort.
- Improves response times for critical issues.
- Enhances reliability of exception detection.
- Allows the team to focus on higher-value tasks.

---

4. Value Addition

Implementing this automation delivers multiple benefits:

Efficiency

- Eliminates repetitive manual steps.
- Reduces the time spent on exception monitoring by more than 80%.

Accuracy

- Minimizes human error, especially in manual comparison of exceptions.

Proactive Monitoring

- Faster detection of new or trending exceptions.
- Immediate automated alerts ensure timely resolution.

Scalability

- Automation can adapt as exception volumes grow.
- Monitoring intervals and logic can be updated without major rework.

Consistency

- Ensures the logic for new/elevated exception detection remains uniform across all shifts and team members.

---

5. Our Approach

To ensure a structured, reliable and future-proof automation, our approach includes:

Step 1 — Build With Mock Data First

Before obtaining access to Dynatrace and the real baseline sheet, we will construct a working end-to-end flow using mock JSON and a mock Excel template. This allows us to:

- Set up the full Power Automate skeleton.
- Implement NEW/ELEVATED detection logic.
- Configure Teams messaging.
- Validate end-to-end flow behavior.

Step 2 — Insert Real Dynatrace Data Inputs

Once Dynatrace access is granted:

- Replace mock API call with the actual Dynatrace API.
- Update Parse JSON schema with the real Dynatrace JSON response.
- Map fields correctly (exception name, count, timestamps).

Step 3 — Integrate the Actual Baseline Sheet

After receiving the team's baseline Excel sheet:

- Align column mappings.
- Apply the team's formula logic.
- Implement real comparison rules.

Step 4 — Finalize NEW and ELEVATED Logic

Using the definitions provided in team discussions:

- "New" → Exception not seen in previous baseline.
- "Elevated" → Count higher than last run, and increasing further after

30-minute re-check.

Step 5 — Enable Teams Notification Logic

Implement branching:

- If **NEW** → Immediate Teams alert.
- If **ELEVATED** → Delay 30 mins → re-run → send alert if increased.
- If **not increased** → no alert.